

MaLiVD: Source Code Vulnerability Localization via Attention-Based Multi-Modal Learning

Enze Dai^{ID}, Shuo Wang^{ID}, Xi Xiao^{ID}, Qing Li^{ID}, *Senior Member, IEEE*, Sheng Wen^{ID}, *Senior Member, IEEE*, and Tianqing Zhu^{ID}, *Member, IEEE*

Abstract—Source code vulnerabilities represent a critical threat to software security, potentially leading to severe consequences such as data breaches and system failures. Traditional static analysis tools, while widely used, suffer from high false positive rates and struggle to adapt to the increasing complexity of modern software. Deep learning-based approaches hold promise for automated vulnerability detection, but they face challenges including limited dataset quality, inadequate feature extraction, and lack of precise vulnerability localization capabilities. To address these limitations, we propose MaLiVD, a novel vulnerability detection method in source code that leverages a multi-modal attention mechanism. MaLiVD not only identifies vulnerability types but also pinpoints the specific lines of code where vulnerabilities are triggered. The model extracts sequential, tree-based, and graph-based features from source code and employs specialized neural networks to learn these diverse representations. By strategically focusing on Points of Interest within the code, MaLiVD effectively prioritizes potentially vulnerable code regions, enhancing both detection accuracy and localization precision. Experimental results show that when compared with eight advanced vulnerability detection models across three large datasets, MaLiVD demonstrates superior vulnerability detection and localization capabilities, maintaining high F_1 scores and localization precision. Particularly on the ReliVul dataset, the F_1 score is improved by 21.82%, and the Top-5 localization accuracy is 18% higher than other methods, with lower false positives across six mainstream vulnerability types, validating MaLiVD's practical application value in real-world environments.

Index Terms—Vulnerability detection, vulnerability localization, multimodal learning, attention mechanism.

I. INTRODUCTION

SOFTWARE vulnerabilities pose a significant and escalating threat to individuals, organizations, and the overall

security of the internet [1]. According to recent statistics [2], the number of newly reported vulnerabilities in 2024 reached 43,757, a 46.7% increase compared to 2023. Among these, 7,777 were classified as high-risk or critical, accounting for 17.8% of the total. These vulnerabilities can lead to data breaches, privacy violations, and significant financial losses for businesses and even nations. Exploiting vulnerabilities in internet-connected devices allows attackers to gain control, potentially leading to data leakage and system failures. With the rapid growth of open-source software, static vulnerability detection in source code has emerged as a crucial first step in the software testing and security lifecycle. The ability to statically identify a large portion of vulnerabilities early in the development process can significantly reduce the human and financial resources required for software testing and maintenance [3]. Consequently, the demand for automated vulnerability detection methods is continuously increasing.

Early vulnerability detection methods in source code focused on scanning source code and analyzing lexical, syntactic, control flow, and data flow information to identify vulnerabilities. Tools like CodeQL [4] and Coverity [5] employ static analysis and rule-based matching to detect system vulnerabilities in program source code. However, these approaches often rely on manually constructed vulnerability patterns by experts. As software complexity increases, the cost and difficulty of manual pattern construction also rise. Moreover, the increasing complexity of newly discovered vulnerability patterns can result in high false positive rates for these static scanning tools, reducing their cost-effectiveness. Other static analysis tools such as Cppcheck [6], Flawfinder [7] and RATS [8] also suffer from the same limitations of manual rule creation and high false positive rates [9].

The rapid advancement of machine learning and deep learning techniques has led to the development of numerous automated methods for vulnerability detection in source code [9], [10], [11], [12], [13], [14], [15]. Some approaches treat code similarly to natural language, encoding words or symbols in the code and then using NLP models for vulnerability detection. However, code differs from natural language, possessing rich structural information and complex data and control flow relationships. To address these differences, other methods [11], [11], [16], [17] extract various code features, such as Abstract Syntax Trees (AST), Control Flow Graphs (CFG), and Data Flow Graphs (DFG), and use machine learning models to learn from these features.

Received 27 April 2025; revised 6 November 2025; accepted 16 March 2026. Date of publication 6 April 2026; date of current version 29 April 2026. This work was supported in part by the Natural Science Foundation of Guangdong Province under Grant 2025A151011946 and in part by the Basic and Frontier Research Project of the Peng Cheng Laboratory under Grant 2025QYA001. The associate editor coordinating the review of this article and approving it for publication was Dr. Yue Zhang. (*Corresponding author: Xi Xiao.*)

Enze Dai is with Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China (e-mail: dez23@mails.tsinghua.edu.cn).

Shuo Wang and Xi Xiao are with Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China, and also with the Peng Cheng Laboratory, Shenzhen 518066, China (e-mail: wangs22@mails.tsinghua.edu.cn; xiaox@sz.tsinghua.edu.cn).

Qing Li is with the Peng Cheng Laboratory, Shenzhen 518066, China (e-mail: liq@pcl.ac.cn).

Sheng Wen is with Swinburne University of Technology, Melbourne, VIC 3122, Australia (e-mail: swen@swin.edu.au).

Tianqing Zhu is with the Faculty of Data Science, City University of Macau, Macau, SAR, China (e-mail: tqzhu@cityu.edu.mo).

Digital Object Identifier 10.1109/TIFS.2026.3681191

Despite these advancements, the effectiveness of existing methods remains unsatisfactory, limiting their practical application. The main challenges include: (1) existing vulnerability datasets suffer from issues such as redundancy, imbalance, low complexity, and small sample sizes, making it difficult to train effective models; [18] (2) the code features extracted by vulnerability detection models often lack specific vulnerability characteristics, hindering the model's ability to learn the underlying principles of vulnerable code; [17] (3) the designed models often lack specificity, rationality, and interpretability, simply adapting models from other domains without proper consideration; [17] and (4) the inability of models to accurately locate the precise location of vulnerabilities [19].

To address the shortcomings of existing vulnerability detection models, this paper proposes a novel vulnerability detection model in source code based on a multi-modal attention mechanism, called *MaliVD*. *MaliVD* is capable of identifying various types of vulnerabilities and locating the specific lines of code where the vulnerabilities are triggered. *MaliVD* extracts sequential, tree-based, and graph-based features from the code and utilizes advanced neural networks tailored to each structure. To enable the model to focus on learning potentially vulnerable code blocks, we employ a heuristic approach by analyzing patterns of various vulnerabilities and extracting code features from locations where vulnerabilities are likely to occur, which is called Points of Interest (PoIs) [19]. By applying masks to the original features, *MaliVD* focuses on statements that may trigger vulnerabilities, significantly improving the accuracy of vulnerability detection and localization. We leverage the attention mechanism within the model to calculate the probability of each line of code triggering a vulnerability, thereby pinpointing the most likely locations of vulnerabilities. While *VulChecker* also leverages PoIs, it operates on compiled LLVM IR. *MaliVD* applies PoI masking to the attention mechanism to reduce the hypothesis space. Unlike the sequence-based *LineVD*, *MaliVD* integrates sequence, structure, and dataflow information to address challenging cases, such as indirect calls.

To comprehensively evaluate *MaliVD* and address the low quality of existing vulnerability datasets, we constructed a real-world vulnerability dataset from the CVE vulnerability platform and open-source code platforms. We ensure the accuracy of labels through keyword filtering and ensemble learning. This dataset not only covers most of the popular CWE types, but also annotates the line numbers where vulnerabilities are triggered. Extensive experiments validate the superior performance of *MaliVD* in vulnerability detection and localization, confirming its practical application value. In summary, the main contributions of this paper are:

- We propose *MaliVD*, a vulnerability detection and localization model in source code based on a multi-modal attention mechanism. We extract a set of vulnerability PoIs features to allow *MaliVD* to focus on the locations where the vulnerabilities may be triggered, improving the accuracy of vulnerability localization.
- We validate the effectiveness of *MaliVD* through extensive experiments. By comparing with eight SOTA methods on three large vulnerability datasets, *MaliVD*

demonstrates the best performance in both vulnerability type detection and vulnerability code line localization, confirming its practical application value.

- We publicly release the source code of *MaliVD* on GitHub,¹ providing a valuable resource for other researchers.

The rest of this paper is organized as follows. Section II discusses the related work. Section III provides the details of *MaliVD*. We discuss the research questions and experiment setup in Section IV. Then, we present the experimental results in Section V. Finally, we conclude the paper in Section VII.

II. RELATED WORK

This section describes the related work [20], [21], [22], [23]. We will first briefly describe traditional vulnerability detection methods, then discuss deep learning-based techniques. Finally, we will explore the performance of current models in vulnerability localization and the challenges that remain in the field. Prior efforts on vulnerability detection and localization either operated on compiled IR or modeled source sequences. For example, *VulChecker* applies PoIs on LLVM IR to prune the search space, while *LineVD* relies on sequence models for line-level localization. Our approach, *MaliVD*, differs by applying PoI masking within the attention mechanism over source code and by jointly incorporating sequence, structural, and dataflow information.

A. Traditional Vulnerability Detection Methods

Early vulnerability detection in source code primarily relied on rule-based static analysis tools, which analyzed lexical structure, syntax, control flow, data flow, and other aspects of source code to identify vulnerabilities. *CPPCheck* [6], *CodeQL* [4], and *Coverity* [5] are typical examples of such tools, and are widely used in the industry. These tools are typically developed by specialized organizations and depend on expert-defined vulnerability patterns and rules. With the advancement of machine learning, some researchers began applying machine learning models to vulnerability detection in source code. Hindle et al. [24] observed similarities between programming languages and natural languages in statistical characteristics, validating the hypothesis of “naturalness” of code. Based on this theory, some studies attempted to extract “language-like” features from code and employ machine learning models for vulnerability detection. For example, *VCCFinder* [25] extracted code features by analyzing code structure and used Support Vector Machines to train on one-hot encoded features for vulnerability identification. However, most current machine learning methods still depend on expert-designed features as input. The increasing complexity of software systems has made these features and patterns more difficult and costly to extract, which limits the efficiency of vulnerability detection.

B. Deep Learning-Based Vulnerability Detection Methods

Deep learning methods do not require expert designed features and can automatically extract deep features from code.

¹<https://github.com/DaichengAuto/MaliVD>

TABLE I
SUMMARY OF EXISTING METHODS

References	Code Features						Models	Loc. ^b
	NCS	AST	CFG	DFG	CPG	CUS ^a		
Russell'18 [10]	✓						CNN/RF	✗
μVulDeePecker [9]						✓	BiLSTM	✗
Devign [11]	✓	✓	✓	✓			GCN/DNN	✗
Li'20 [12]	✓						CNN	✗
Zagane'20 [13]	✓						DNN	✗
Funded [14]					✓		GNN/GRU	✗
SySeVR [15]	✓					✓	BiRNN	✗
IVDetect [16]	✓	✓	✓			✓	GRU/TreeLSTM	✓
BGNN4VD [26]					✓		GNN/GRU	✗
Vuldeelocator [27]	✓					✓	BiRNN	✓
ReVeal [1]					✓		GCN/DNN	✗
VulCNN [28]	✓						CNN	✗
ReGVD [29]	✓				✓		Res-GCN	✗
MVD [30]	✓		✓	✓			FS-GNN	✗
LineVul [31]	✓						Transformer	✓
LineVD [32]	✓		✓	✓			Transformer/GAT	✓
CPVD [33]					✓		GAT/ConvPool	✗
VulChecker [19]						✓	MP-GNN	✓
DeepVD [17]	✓	✓				✓	GRU/GCN	✗
EnGS2F [34]		✓	✓	✓			GGNN	✗
MGVD [35]	✓	✓	✓				GAT/CNN	✗
DetectVul [36]	✓						Self-Attention	✓
SecureQwen [37]	✓						CodeQwen	✗
MaliVD	✓	✓			✓	✓	Hybrid/ATT	✓

^aCUS: code representations customized by researchers.

^bLoc.: whether the method can perform vulnerability localization.

In recent years, many studies have focused on applying deep learning to vulnerability detection, achieving notable results, as shown in Table I.

Early efforts primarily focused on learning from Natural Code Sequences (NCS) using sequence-based models such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) [9], [10], [12], [13], [28]. However, these models overlook the structural information in code, making it difficult for them to learn and leverage various contextual semantic information. To address these limitations, subsequent studies explored new structured code representations, include AST [11], CFG [15], DFG [15], and Program Dependence Graphs (PDG) [34]. Advanced code representations include Code Property Graphs (CPG) [11], [33]. CPG integrates the edge information of AST, CFG, and PDG, containing more comprehensive information, and is widely used in current methods. In terms of models, some studies have utilized Gated Recurrent Units (GRU) [16], Tree-structural Long Short-Term Memory (TreeLSTM) [16], [17], Graph Convolutional Networks (GCN) [1], [11], and Gated Graph Neural Networks (GGNN) [14], [16], [26], [34] to learn code representations.

Recently, many studies focused on improving neural network models, achieving better results, such as adding residual networks to the GCN [29], incorporating attention mechanisms [35], improving the node and edge learning process in the GNN to be sensitive to flow information [30], and using message-passing neural networks [19] for code analysis. others combined different code representations, using different models to learn features of different structures (e.g., combining sequence models and graph models). However, these methods often apply models from other domains to vulnerability

detection without an in-depth analysis of vulnerability patterns and consideration of whether the model is truly suitable for learning these code representations [17].

C. Deep Learning-Based Vulnerability Localization Methods

Although deep learning-based vulnerability detection techniques have demonstrated feasibility in experiments, most methods can only provide coarse-grained results, but are unable to locate the vulnerabilities. Initially, researchers explored the use of neural network explanations to determine vulnerability locations. Zou et al. [38] extracted a series of rules from vulnerability detection model decisions to identify which statements were more important for vulnerability detection results. Additionally, Li et al. [16] proposed an explainable vulnerability detector, IVDetect, which uses GNNExplainer [39] to provide fine-grained explanations of vulnerability-related statements and support the localization of vulnerable statements, but struggles to achieve accurate vulnerability localization. With the widespread application of Graph Attention Networks (GAT) [40], many studies [33], [35] used GAT and other attention mechanisms [36], [41] for vulnerability localization but have still failed to achieve satisfactory results. However, the explanation of neural networks does not incorporate vulnerability pattern analysis, making it difficult to give explanations related to the vulnerability.

Recently, methods such as Vuldeelocator [27] and VulChecker [19] attempted to perform vulnerability localization through compiled intermediate code (e.g., LLVM). They leveraged intermediate code representations and attention mechanisms for vulnerability localization. However, these methods relied on compiled code information, making them impractical in real software development. With the successful application of LLMs in various tasks, models such as LineVul [31] and LineVD [32] based on CodeBERT [42] and SecureQwen [37] based on CodeQwen [43] are proposed for vulnerability detection and localization, but the accuracy of these methods is still not ideal. This is because code is not just text but contains rich structural information. Directly applying LLMs to process source code can lead to the loss of structured features. Additionally, vulnerabilities often manifest as subtle feature changes, making it difficult for LLMs, which are trained on complex information and emphasize global features, to capture these small but critical features.

D. Current Problems With Vulnerability Detection Models

Based on the analysis of the performance of existing work in vulnerability detection, their problems are as follows [44]: reliance on low-quality vulnerability datasets, inappropriate feature selection, inappropriate model design, and difficulty in localizing vulnerability locations. These challenges highlight the need for improved methodologies in vulnerability detection research.

To address the inadequacies of existing vulnerability detection models, this paper introduces MaliVD, a novel vulnerability detection method in source code based on a multi-modal attention mechanism. MaliVD is capable of identifying various types of vulnerabilities and precisely localizing

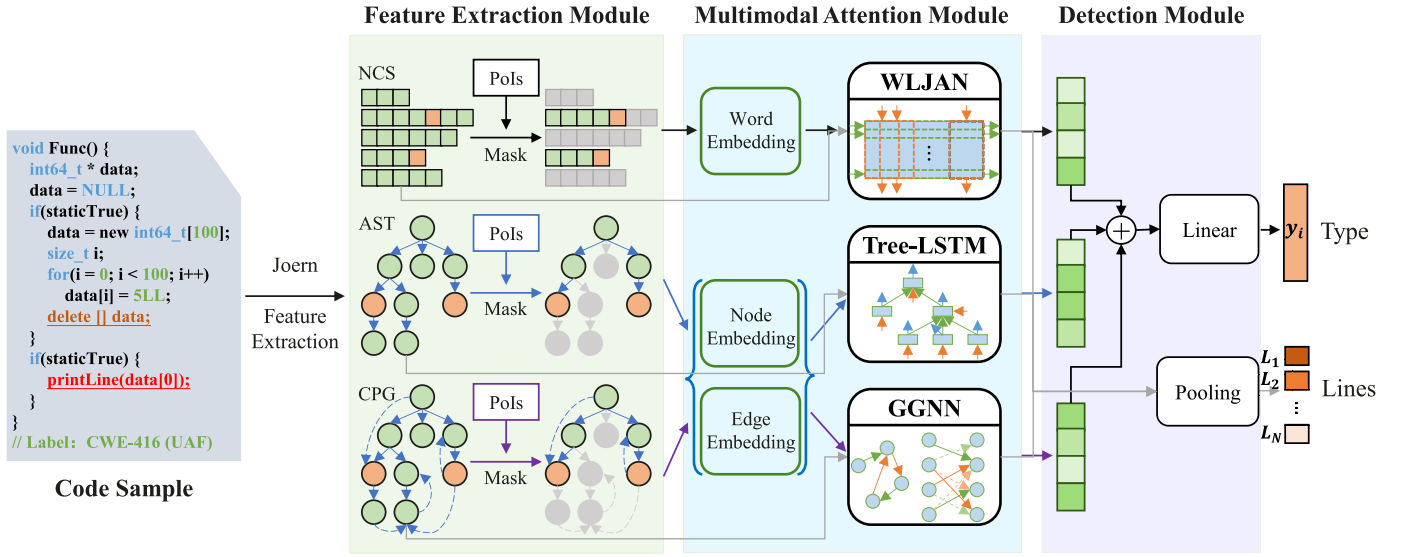


Fig. 1. The overall structure of MaliVD.

them to the specific lines of code that trigger these vulnerabilities. MaliVD extracts three types of features from the code: sequential, tree-based, and graph-based, and employs advanced neural networks tailored to each structure for learning. By applying masking to the original features, MaliVD focuses on statements likely to trigger vulnerabilities, thereby significantly enhancing the accuracy of both vulnerability detection and localization.

III. SYSTEM DESIGN

This section describes the architecture of our proposed method. We first show the model overview in Section III-A, then discuss the data preprocessing and feature extraction phase in Section III-B. Section III-C details the entire framework of MaliVD, including the internal model structure design, as well as how it performs vulnerability detection and localization.

A. Model Overview

This paper proposes a novel vulnerability detection and localization model based on multi-modal attention, named MaliVD (Multimodal Attention-based Line-level Vulnerability Detector). The architecture of MaliVD is shown in Figure 1.

As discussed in Section II, source code can be represented using three structural code representations: sequential, tree-based, and graph-based representations. We employ sequence-based, tree-based, and graph-based neural networks to learn corresponding representations. After training individual models for each representation, an ensemble model is employed to integrate the results. Furthermore, to achieve fine-grained vulnerability localization, we first design a set of vulnerability PoIs features, which are statements likely to trigger vulnerabilities. These PoIs are designed through analyzing the characteristics of various CWE vulnerabilities. Then, for sequential, tree, and graph feature structures, we apply masking to statements and paths after the PoIs and other statements and paths without PoIs. Both original and masked

features are fed into the three models respectively. The outputs of the two parts are then combined and fed to classifiers for each vulnerability type. Moreover, by leveraging the attention weights learned during the model training, we compute the probability that each line of code might trigger a vulnerability. Thus, we are able to localize the potential positions of the vulnerabilities.

B. Feature Extraction Module

1) *Data Preprocessing*: Source code samples may contain comments, strings, user-defined variable names, and function names. These elements need to be cleaned and standardized before feature extraction and model input. For a code sample, we first remove comments (both single-line and multi-line), blank lines, and extra spaces using regular expressions to reduce the impact of irrelevant information. In addition, constant values (e.g., numbers, strings) within the code are replaced with unified placeholders. For example, all numbers are replaced with “NUM” and all strings with “STR”. This helps to prevent the model from overfitting specific constant values. Moreover, user-defined function and variable names are renamed sequentially to a standardized format, such as “FUNC0”, “FUNC1”, “VAR0”, and “VAR1”. This eliminates noise caused by identifier names, enabling the model to focus on the code’s logical structure. Through these steps, we efficiently clean and standardize code samples, providing high-quality data for feature extraction and model training.

2) *Feature Extraction*: Although source code is essentially a sequence of tokens, it has rich structural information, and can be transformed into tree-based and graph-based representations. This paper focuses on vulnerability detection in C/C++ code. We extract ASTs and CPGs using Joern [45], and combine them with the NCS as three code representations. Additionally, we conduct statistical analysis based on the CWE platform [46] and other online resources. Based on the popularity and severity of different vulnerabilities, this paper focuses on high-frequency and high-impact vulnerabilities, classifying them into six categories: buffer overflows, value

TABLE II
CWE CATEGORIES, EXAMPLES, AND CORRESPONDING POI STANDARDS

Vulnerability Category	CWE IDs	Poi Standards
Buffer Overflows	CWE-119, CWE-121, CWE-122, CWE-125, CWE-119	Any read or write operation to local or dynamically allocated memory
Value Calculation Vulnerabilities	CWE-190, CWE-369, CWE-682	Variables that undergo operator calculations or function calls
Resource Management Vulnerabilities	CWE-120, CWE-399, CWE-400, CWE-404, CWE-415, CWE-416, CWE-476, CWE-908, CWE-909	Pointer variables, memory allocation, and access instructions
Input Validation Vulnerabilities	CWE-20, CWE-89, CWE-134	Any variables and access statements from user input
Permission Management Vulnerabilities	CWE-255, CWE-264, CWE-269, CWE-284, CWE-287, CWE-863	Verification statements before operations involving files, network ports, databases, command execution, etc.
Command Execution Vulnerabilities	CWE-77, CWE-78, CWE-88	Statements executing system commands and related variables

calculation vulnerabilities, resource management vulnerabilities, input validation vulnerabilities, permission management vulnerabilities, and command execution vulnerabilities. These six categories cover most of the CWEs. Some example CWEs are shown in Table II.

To extract Vulnerability location features, i.e., PoIs, we analyzed each vulnerability category in detail and formulated corresponding Poi standards, as listed in Table II.

Take buffer overflow vulnerabilities as an example, common CWEs are CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer), CWE-121 (Stack-based Buffer Overflow), and CWE-122 (Heap-based Buffer Overflow). These vulnerabilities typically occur when a program fails to handle memory boundaries correctly. These vulnerabilities may cause data to overflow into adjacent memory areas, thereby causing security problems. Therefore, we define any read or write operation to local or dynamically allocated memory as a Poi standard, including function calls such as `memcpy`, `malloc`, and `free`. Similarly, we define PoIs for the other five types of vulnerabilities. The Poi standards are shown in Table II.

Additionally, code is essentially a sequence of tokens. To understand the quantity of these tokens after preprocessing, we statistically categorize and count them. This includes 98 keywords, 31 symbols, 928 system functions, 100 standardized function names, 200 standardized variable names, 26 node attribute features, and 6 Poi features, totaling 1389 tokens. For edge information in tree and graph representations, we extract AST, CFGs, and DFGs edge types, along with 14 inter-node call relationships.

Combining this node and edge information with the NCS, AST, and CPG results in three different code representations. It is worth noting that, for PoIs nodes in the features, we derive three masked features by setting the other parts as masks. In specific, for code sequence features, if one node of a code line is identified as a PoIs node, this line's PoIs node and the nodes before it are reserved, others are padded with zeros. For AST, if one node in the tree is marked as a PoIs node, all paths from the root to this node are potentially vulnerable and are reserved. Other nodes which are not on these paths are padded with zeros. Similarly, for CPG, all paths from the root to the PoIs node are reserved, and all other parts are masked. For the unmasked parts, this operation actually gets a sub-sequence, a sub-tree, and a sub-graph. The purpose is to make the model focus more on the code that is

likely to trigger vulnerabilities, which is more conducive to vulnerability detection and localization.

Through these detailed node and edge features, the model can comprehensively capture the syntax, control flow, data flow, and semantic information of the code. These features enhance the model's ability to understand the code and detect vulnerabilities. For instance, keywords and symbols help the model recognize the basic structure of the code; system functions and calling relationships help identify potential memory management problems and function call exceptions; standardized variables and functions reduce the noise; and the tagging of PoIs nodes and paths contributes significantly to the model's ability to locate vulnerabilities. By combining these features, the model can effectively identify and locate vulnerabilities in complex and varied code environments.

C. Multimodal Attention Module

This section details the architecture of the multi-modal attention model, which leverages diverse code representations for vulnerability analysis. Given a code sample, we derive six feature spaces after feature processing and masking: NCS, AST, CPG, and their corresponding masked versions. To learn these representations, we employ three types of neural network models: the Word-Label Joint Attention Network (WLJAN) [47], the TreeLSTM [48], and the GGNN [49]. We now elaborate on each of these models and their learning processes.

1) *Sequential Structure Model*: The WLJAN model in this paper is an adaptation of the model proposed by Xiao et al. [47]. While the original model was designed for network traffic, we argue that serialized code data, like network traffic, exhibits structured sequential features. As highlighted in [47], traditional neural networks, often used for image and text data, are not optimally suited for traffic data, which lacks the spatial characteristics of images or the large vocabulary of text, and instead consists of only 256 bytes. Similarly, processed code nodes in our case have only 1289 features, with many nodes being infrequently used. Thus, we modified their model, resulting in a critical component of our approach for processing sequential code features.

Specifically, we treat nodes within the code as bytes in network traffic. Each word and label is initially embedded into a vector. Assuming a code sequence of length N_N and K labels, the embedding process produces N_N word vectors, denoted as $W = \{w_i + Q_i\}_{i=1}^{N_N}$, and K label vectors, $Y = \{y_i\}_{i=1}^K$.

Here, w_i represents the embedding vector of the word at position i and y_i is the embedding of the i -th label, and Q_i is the positional encoding of position i . We then compute the similarity matrix between words and labels using cosine similarity and get a similarity matrix $G = [G_{(i,j)}]$, where $i = 1, \dots, N_N, j = 1, \dots, K$. It is then fed into a neural network to learn word and label attention.

To better capture sequential code information, we employ GRU [50] as the core network. For the t -th word, assuming its input is x_t , the output of GRU is $h_t \in \mathbb{R}^{f_N}$, where f_N is the dimension of the output layer, we apply a weighted connection and pooling operation to each output h_t , resulting in a word attention score $\text{ATT}_t^{(N)}$ calculated as follows:

$$\text{ATT}_t^{(N)} = \tanh(\text{MP}(\text{FC}([h_{t1}, \dots, h_{tf_N}]))), \quad t = 1, \dots, N_N, \quad (1)$$

where MP and FC stand for Max Pooling and Fully Connected layers respectively, $\tanh$ denote the $\tanh$ activation function.

As mentioned, the sequential features include both the original and the masked sequence features. Both sequences are processed as described above, resulting in their corresponding attention scores. Let the attention scores of unmasked and masked sequences be $\alpha_t^{(N)}$ and $\beta_t^{(N)}$, respectively. These two scores are summed, and the result is used to weight the word vectors, obtaining the word attention vectors. For label attention and generating the overall code representation vector, we follow the same method as in the original paper. Finally, we get the code representation as shown in equation (2).

$$h_N = \sum_{t=1}^{N_N} (\alpha_t^{(N)} + \beta_t^{(N)}) x_t, \quad (2)$$

where $h_N \in \mathbb{R}^E$ represents the output of the WLJAN model, which is the high-level representation of the sequential code of dimension E .

2) *Tree Structure Model*: For tree-structured representations like ASTs, a tree-based neural network is more suitable. Here, we use Child-sum TreeLSTM [48]. Unlike the traditional LSTM, the parent node's information comes from all its children. This approach sums the hidden states of all children before updating the hidden state of the parent, without any restriction on the number of children. Traditional LSTMs can be viewed as a special case of TreeLSTM where all nodes have at most one child.

After bottom-up computation, we can obtain each AST node's output vector. To improve the performance of vulnerability detection and localization, we add an attention mechanism to compute the weight of each AST node. For the j -th node, assuming the input vector is x_j , the output through TreeLSTM is h_j . This attention is calculated as follows:

$$\text{ATT}_j^{(T)} = \sigma(\text{FC}([h_j^T \| x_j])) \odot \tanh(\text{FC}([h_j^T \| x_j])), \quad j = 1, \dots, N_T, \quad (3)$$

where $\|$ represents the vector concatenation. The σ activation function normalizes the input, selecting nodes that contribute most to the neural network. The $\tanh$ activation function determines whether a node has a positive or negative impact, accelerating convergence and preventing vanishing gradients.

Therefore, $\text{ATT}_j^{(T)}$ is the attention score of the j -th node. As with sequential features, tree-based features have original and masked versions. Both features are processed using TreeLSTM. Let the attention scores of unmasked and masked tree features be $\alpha_j^{(T)}$ and $\beta_j^{(T)}$, respectively. Summing these scores and weighting the node vectors, the final tree-based representation is:

$$h_T = \sum_{j=1}^{N_T} (\alpha_j^{(T)} + \beta_j^{(T)}) x_j, \quad (4)$$

where $h_T \in \mathbb{R}^E$ is the output of the TreeLSTM model, which is also the high-level representation of the tree-structured code of dimension E .

3) *Graph Structure Model*: Program dependency relationships such as data flow and control flow can be represented as graphs, $G = (V, E)$, where $v \in V$ stores D -dimensional feature vectors and $e \in E$ stores $D \times D$ -dimensional adjacency matrices. We use GGNN [49] to learn the extracted CPG features.

Assume that the input of node v is x_v , and the node representation after GGNN learning is h_v . Similar to Eq. (3), we also introduce a graph attention mechanism to select the most relevant nodes for the overall graph output. The attention score $\text{ATT}_v^{(G)}$ of node v is:

$$\text{ATT}_v^{(G)} = \sigma(\text{FC}([h_v^T \| x_v])) \odot \tanh(\text{FC}([h_v^T \| x_v])), \quad v \in V. \quad (5)$$

To output a feature vector instead of a feature map, we sum the node vectors in a weighted manner. Both the original and masked features are computed separately, and their respective attention scores are $\alpha_v^{(G)}$ and $\beta_v^{(G)}$. The final graph representation vector is:

$$h_G = \sum_{v \in V} (\alpha_v^{(G)} + \beta_v^{(G)}) x_v, \quad (6)$$

where $h_G \in \mathbb{R}^E$ represents the output of the GGNN model, and is also the high-level representation of the graph-structured code of dimension E .

D. Detection Module

This section includes two tasks: vulnerability classification and vulnerability localization.

1) *Vulnerability Classification*: We first concatenate the three high-level vectors obtained from the models above by $h = [h_N \| h_T \| h_G]$. $h \in \mathbb{R}^{3E}$ is the final code representation vector. Then we use a linear layer to connect h to the K vulnerability labels. Finally, through softmax, we get the classification result $\hat{y}$.

We use the *Focal Loss* [51] as our classification model's loss function because it performs well on imbalanced datasets and allows for better learning from hard samples. The *Focal Loss* is computed as follows:

$$\begin{aligned} \mathcal{L}_{\text{pac}} &= FL(y, \hat{y}; \alpha, \gamma) \\ &= -\alpha [1 - \text{softmax}(\hat{y}_y)]^\gamma \log(\text{softmax}(\hat{y}_y)), \end{aligned} \quad (7)$$

where y is the ground truth, $\hat{y}_y$ is the predicted probability for the true label, α and γ are the hyperparameters of the *Focal Loss*.

Loss. We set α to 1 minus the fraction of samples for that label, and γ to 2.

2) *Vulnerability Localization:* The model's prediction of the vulnerability trigger locations is obtained by analyzing the attention scores of each node, as computed using equations (1), (3), and (5) for sequential, tree, and graph neural networks, respectively. Given that a single line of code can correspond to multiple AST or CPG nodes, we first sum the sequential attention, the tree attention, and the graph attention of all the nodes in the same line to obtain the line-level attention scores. We define $\text{MASK}(\cdot)$ as a masking function for attention scores similar to Subsection III-B.1. This ensures that we retain the code lines where PoIs exist because only those locations could be the trigger points of vulnerabilities. Suppose the summed attention score after masking is $\text{ATT} \in \mathbb{R}^M$, where M is the total number of code lines in the sample. By masking and normalizing ATT , we obtain the probabilities L_i of each line potentially triggering a vulnerability, as described in the following equation:

$$L = [L_1, \dots, L_M] \\ = \text{softmax}(\text{MASK}(\text{ATT})). \quad (8)$$

The vulnerability detection module classifies a code sequence into a specific vulnerability category, using *Focal Loss* as the loss function. Similarly, in the vulnerability localization module, the model predicts the probability of each line of code being a vulnerability trigger point, and we use *Focal Loss* as the loss function for the localization module, analogous to equation (7), where y and $\hat{y}_y$ now represent the true line number that triggers the vulnerability and the model's prediction probability for that true line number, respectively. Three attention scores are bounded within the interval $(-1, 1)$ by activation function σ and $\tanh$, thereby aligning them to a consistent scale. By back-propagating the gradients from both the vulnerability type and the vulnerability line number, the model is able to identify vulnerabilities and accurately locate the specific trigger points of those vulnerabilities.

IV. EXPERIMENTAL DESIGN

In this section, we present our research questions, experimental datasets, baseline methods, and evaluation metrics.

A. Research Questions

To systematically evaluate our proposed method, we formulate the following research questions, each targeting specific aspects of the model's performance and capabilities.

RQ1: How effective is MaLiVD for function-level vulnerability detection? This question assesses MaLiVD's detection capabilities at the function level, a standard evaluation granularity for vulnerability detection tools.

RQ2: How effectively does MaLiVD localize vulnerabilities at the code line level? Precise line-level vulnerability identification is crucial for efficient code remediation.

RQ3: How does MaLiVD's detection performance vary across vulnerability categories? This question investigates MaLiVD's performance in detecting six major vulnerability

categories, examining potential performance disparities influenced by vulnerability complexity.

RQ4: What drives MaLiVD's superior performance in vulnerability detection and localization? Through ablation studies, we systematically evaluate the contribution of each architectural component in MaLiVD.

B. Experiment Setup

This section describes the experimental setup used to rigorously evaluate MaLiVD, encompassing details regarding the datasets, baseline methods, and implementation procedures. We aim to provide a transparent and reproducible methodology for our experimental analysis.

1) *Datasets:* To comprehensively evaluate MaLiVD's vulnerability detection and localization capabilities, we employ three distinct datasets: the public Devign and DiverseVul datasets, alongside our newly created ReliVul dataset. Devign [11] and DiverseVul [18] provide function-level samples and are widely used.

To better evaluate fine-grained vulnerability localization, we present ReliVul (Real-world Line-level Vulnerability dataset), a new dataset created specifically for this task. The construction of ReliVul follows a similar methodology to DiverseVul, leveraging information from NVD [52] and Mend [53] vulnerability databases. We mined vulnerable code samples from open-source projects using commit history, focusing on patches that resolve known vulnerabilities. Keyword filtering sourced from vulnerability databases is then applied to recall candidate vulnerable lines from the code diffs. For the CWE-PoI pair label, we focus on the prevalent and high-impact vulnerability categories listed in Table II. Candidate lines need to match these categories to remain eligible. These lines are cross-validated using ensemble learning of static analysis tools, trained ML detectors, and an LLM checker to ensure the accuracy of CWE labels. Specifically, we employ CodeQL [4] and CppCheck [6] as static analysis tools, LineVul [31] as the ML detector, and GPT-4 as the LLM checker. Disagreement lines after ensemble learning will be discarded.

In summary, we use our designed vulnerability PoIs, alongside the code revision history, to precisely annotate the line of code triggering the vulnerability. ReliVul includes not only function-level information but also provides precise line-level vulnerability trigger points, enabling a more thorough evaluation of both vulnerability detection and localization capabilities. Furthermore, ReliVul is labeled with CWE, ensuring comprehensive ground-truth information.

2) *Baseline Methods:* To provide a thorough and fair comparison, we evaluate MaLiVD against eight state-of-the-art baseline methods that span a range of techniques. These methods include: Code2Vec [54], Devign [11], ReVeal [1], Funded [14], MGVD [35], IVDetect [16], LineVul [31], and CppCheck [6]. All baseline methods are used for the function-level vulnerability detection task. IVDetect, LineVul, and CppCheck, which have line-level detection capabilities, are also evaluated for the vulnerability localization task.

We report parameter scale on the same hardware to assess storage overhead. When no pre-trained models are considered, MaLiVD - 9.1Million (M) parameters; Code2Vec - 27.5M;

TABLE III
EXPERIMENTAL RESULTS OF VULNERABILITY DETECTION ON DEVIGN, DIVERSEVUL AND RELIVUL DATASETS FOR EACH MODEL

Models	Devign				DiverseVul				ReliVul			
	Acc.	P.	R.	F_1 .	Acc.	P.	R.	F_1 .	Acc.	P.	R.	F_1 .
Code2Vec	0.5932	0.5587	0.5823	0.5703	0.6578	0.4078	0.4810	0.4414	0.5571	0.4452	0.5318	0.4847
Devign	0.5966	0.5324	0.5889	0.5592	0.7212	0.4156	0.5137	0.4595	0.5643	0.3957	0.5759	0.4691
ReVeal	0.6023	0.5401	0.5694	0.5544	0.7281	0.4375	0.5383	0.4827	0.6050	0.4786	0.6693	0.5581
IVDetect	0.6177	0.5523	0.6288	0.5881	0.7828	0.5638	0.6119	0.5869	0.6339	0.5713	0.6113	0.5906
Funded2021	0.6098	0.5876	0.6138	0.6004	0.8215	0.5189	0.7178	0.6024	0.6022	0.4613	0.5854	0.5160
LineVul2022	0.6159	0.4903	0.6390	0.5549	0.7548	0.5725	0.6254	0.5978	0.6250	0.4968	0.5665	0.5294
MGVD	0.6569	0.5904	0.6291	0.6091	0.7344	0.6256	0.6837	0.6534	0.6189	0.5980	0.5964	0.5972
MaliVD	0.7533	0.6837	0.7283	0.7053	0.8178	0.6671	0.7128	0.6892	0.7143	0.6865	0.7737	0.7275
Improvement	0.0964	0.0933	0.0893	0.0962	-0.0037	0.0415	-0.005	0.0358	0.0804	0.0885	0.1044	0.1303

Devign - 4.3M; ReVeal - 11.7M; Funded - 8.5M; MGVD - 16.3M; IVDetect - 1.5M; LineVul - 8.8M. CppCheck is a static analysis tool. Despite using three subnetworks, PoI masking shrinks the hypothesis space, and at inference, the subnetworks can run in parallel across multiple GPUs, so wall-clock time is well below a threefold increase.

These baseline methods spans a diverse range of techniques from graph-based neural networks to sequence models and static analysis tools, and also covers both function-level and line-level approaches, ensuring a comprehensive and robust evaluation of the proposed MaliVD framework.

3) *Implementation*: To ensure rigorous and reliable results, each method was evaluated using a 10-fold cross-validation strategy. The datasets are randomly split into ten equal partitions, and in each iteration, eight partitions are used for training, one for validation, and one for testing. All experiments were performed on a server equipped with 128 GB of memory, an Intel Xeon Silver 4210 CPU, and an NVIDIA 2080 Ti GPU. To guarantee fairness and isolate the effects of our approach, we have anonymized all custom variables, types, and function names in the code samples and performed other essential standardizations.

For parameter tuning, we use a grid search strategy on the validation sets to identify optimal values. Specifically, we combine random search and grid search to select optimal parameters for each algorithm to ensure they reach their best performance. All the methods are trained using the Adam optimizer with an initial learning rate of 1e-3. The learning rate is decayed using a cosine decay scheduler for better convergence. Early stopping is used based on validation set performance to avoid overfitting. The batch sizes are selected based on hardware limitations and empirical analysis, ranging from 32 to 128 for different models.

4) *Evaluation Metrics*: The vulnerability detection task is framed as a multi-class classification problem. Therefore, we use common metrics such as Precision (P), Recall (R), F_1 -score (F_1), and Accuracy (Acc) to provide a comprehensive view of the performance. To evaluate the localization capabilities of our proposed MaliVD, we employ Top-k accuracy and Initial False Alarm (IFA) metrics [31]. Top-k accuracy assesses the model's effectiveness by gauging the proportion of instances where at least one of the top-k recommended lines harbors actual vulnerable code. The IFA metric quantifies the number of false positives a security analyst must

scrutinize before encountering the first genuine vulnerable line of code.

V. EXPERIMENTAL RESULTS

In this section, we present the experimental results with respect to our four research questions.

A. RQ1: How Effective Is MaliVD for Function-Level Vulnerability Detection?

To evaluate MaliVD's effectiveness in function-level vulnerability detection, we conduct comprehensive experiments across three datasets: Devign, DiverseVul, and ReliVul. The experimental results are shown in Table III.

1) *Devign Dataset Analysis*: On the Devign dataset, MaliVD achieves an F_1 score of 0.7053, surpassing the second-best model (MGVD) by approximately 15%. Despite the relatively small size of the dataset, its manually curated labels ensure high accuracy. The results reveal that single-model approaches, such as Code2Vec and LineVul, achieve F_1 scores below 60%, highlighting their limitations in capturing complex code structures. In contrast, multi-feature or multi-model methods like IVDetect, Funded, and MGVD perform better due to their ability to integrate structural and semantic information. Notably, MaliVD's performance exceeds the second-best method by over 10% across all metrics, demonstrating its robustness in real-world scenarios. This advantage stems from its multi-modal attention mechanism, which integrates sequence, tree, and graph representations, enabling it to effectively handle noisy code environments. Additionally, the PoIs features significantly enhance vulnerability detection by guiding the model to focus on potential vulnerability-prone code segments.

2) *DiverseVul Dataset Analysis*: The DiverseVul dataset, sourced from security platforms and open-source repositories, presents a more challenging scenario due to its class imbalance and noise. According to our statistics, only 5.73% of the functions in this dataset are labeled as vulnerable. While some models, such as Funded, achieve high accuracy (82.15%), they often do so by favoring the majority class, leading to high false negatives. In contrast, MaliVD achieves an F_1 score of 68.92%, outperforming the second-best model (MGVD) by 5.48%. This improvement underscores MaliVD's ability to handle imbalanced datasets effectively. By leveraging multi-modal representations and focusing on PoIs, MaliVD

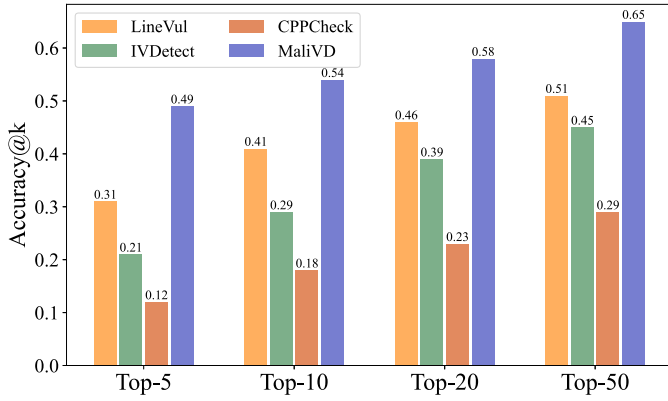


Fig. 2. Results of Top-k accuracy of each methods on the ReliVul dataset.

maintains balanced precision and recall, even when faced with noisy and incomplete data. The use of Focal Loss further enhances its performance by prioritizing the learning of rare vulnerability patterns, which are often overlooked by other methods.

3) *ReliVul Dataset Analysis*: The ReliVul dataset, constructed in this study, offers a rigorous testbed due to its high code authenticity, complex vulnerabilities, and precise annotations of CWE types and trigger locations. Compared to other datasets, ReliVul poses a greater challenge for vulnerability detection. Interestingly, most models perform better on ReliVul than on DiverseVul, indirectly validating the latter's issues with label accuracy and class balance. MaliVD excels on ReliVul, achieving a 12.68% improvement in accuracy and a 21.82% increase in F_1 score compared to the second-best method. Its superior performance highlights the effectiveness of its multi-modal attention mechanism and PoIs-based feature extraction. Furthermore, the integration of Focal Loss ensures that MaliVD prioritizes learning from underrepresented vulnerability classes, enabling it to achieve strong macro-averaged metrics. This capability reinforces its practical applicability in real-world scenarios where vulnerability patterns are sparse and complex.

4) *Conclusion for RQ1*: MaliVD demonstrates exceptional function-level vulnerability detection capabilities across all three datasets, particularly excelling on the challenging ReliVul dataset. Its success can be attributed to the integration of multi-modal code representations, the attention mechanism focused on PoIs, and the use of Focal Loss to address class imbalance. These innovations enable MaliVD to achieve high accuracy and robust generalization.

B. RQ2: How Effectively Does MaliVD Localize Vulnerabilities at the Code Line Level?

To evaluate the line-level vulnerability localization performance of MaliVD, we conduct experiments on the ReliVul dataset, which provides ground truth labels for both CWE types and vulnerable code locations within real-world projects. We compare MaliVD against LineVul, IVDetect, and CPPCheck using Top-k accuracy and Initial False Alarms (IFA) as evaluation metrics.

1) *Top-k Accuracy Analysis*: Figure 2 illustrates the Top-k accuracy ($k = 5, 10, 20, 50$) of MaliVD, LineVul, IVDetect,

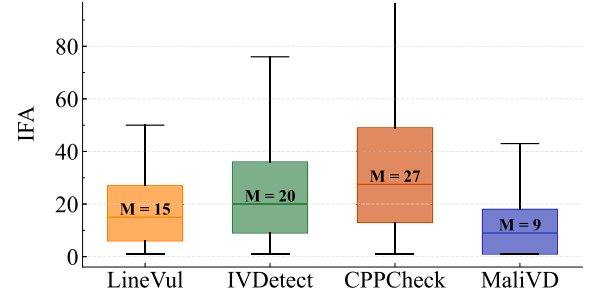


Fig. 3. The average IFA distribution of each method on the ReliVul dataset.

and CPPCheck on the ReliVul dataset. The results demonstrate that MaliVD consistently outperforms the other three methods across all Top-k metrics. Notably, MaliVD achieves a Top-5 accuracy of 0.49, surpassing LineVul (the next best performer) by 0.18. Similarly, at Top-10, MaliVD attains an accuracy of 0.54, exceeding LineVul by 0.13. These results suggest that MaliVD excels not only in identifying potentially vulnerable code regions but also in accurately pinpointing the precise location of vulnerabilities.

For sequence-based LineVul, it processes code blocks as sequences to predict vulnerable lines. Although LineVul can perform vulnerability localization and give explainable results, it ignores the information of code structure, and its performance on top-k is lower than that of MaliVD. As for IVDetect, despite GCN's ability to capture structural information, IVDetect's performance lags behind LineVul and MaliVD. This could be attributed to information loss during the PDG construction or over-smoothing issues in large code graphs. In addition, using GNNExplainer may not provide high localization accuracy. CPPCheck, a static analysis tool relying on predefined rules and patterns, exhibits the lowest Top-k accuracy and tends to produce numerous false positives. In contrast, MaliVD leverages a multi-modal attention mechanism to deeply understand code semantics and structure. By integrating sequential, tree-based, and graph-based representations of source code, MaliVD captures a more comprehensive view of the code. Furthermore, MaliVD strategically focuses on PoIs within the code, guiding the model to prioritize learning from regions prone to vulnerabilities. MaliVD's Top-5 accuracy of nearly 50% implies that security analysts using MaliVD have nearly a 50% chance of finding the vulnerability within the first five lines inspected. This efficient localization significantly reduces manual effort and accelerates vulnerability remediation.

2) *Initial False Alarms Analysis*: The IFA metric quantifies the number of non-vulnerable code lines a security analyst must examine before reaching the actual vulnerability. Figure 3 presents the average IFA distribution of each method on the ReliVul dataset.

The experimental results reveal a significant advantage for MaliVD in terms of the IFA metric, with a median IFA of only 9. This value is considerably lower than LineVul (15), IVDetect (20), and CPPCheck (27). This implies that, on average, security analysts using MaliVD only need to inspect approximately nine non-vulnerable lines before identifying

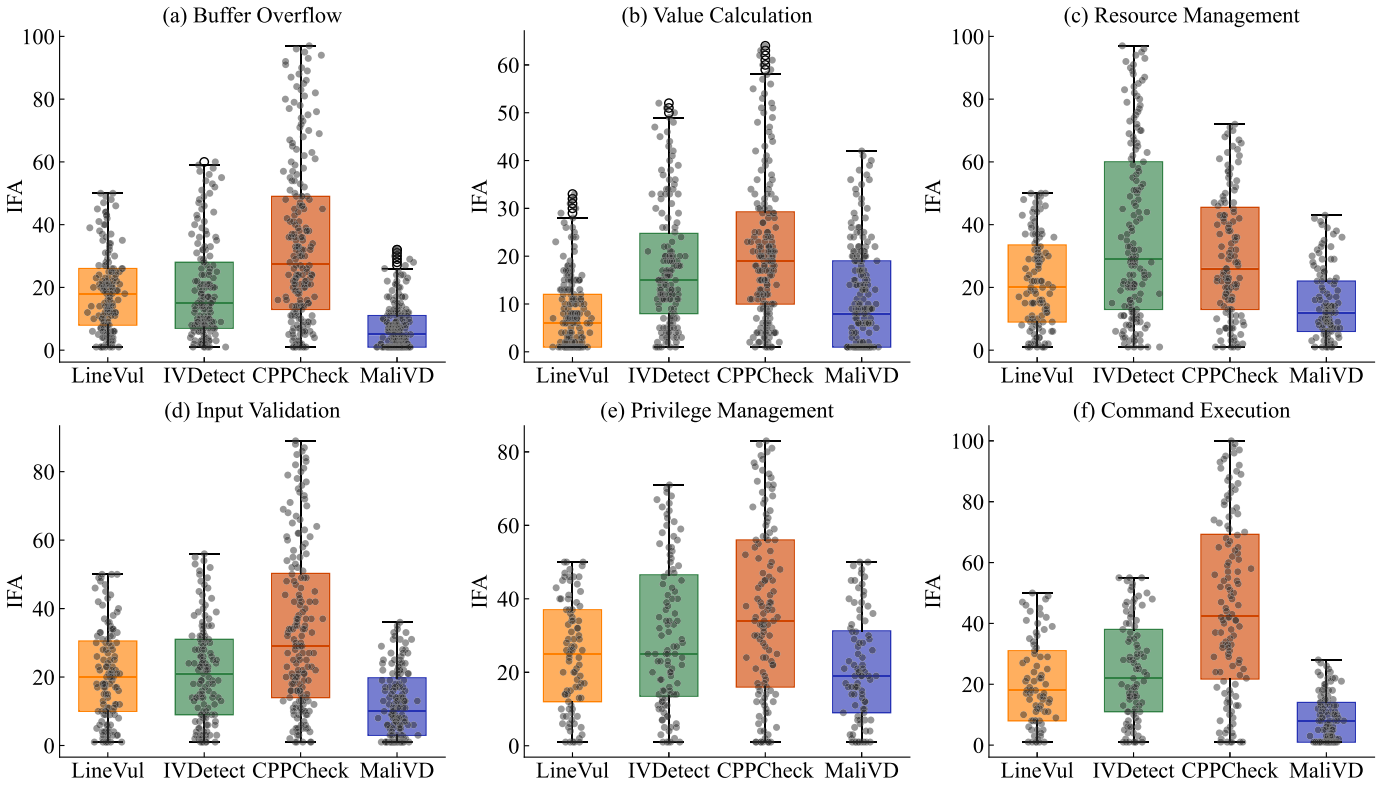


Fig. 4. Comparison of IFA value distributions across six vulnerability types for different models.

the actual vulnerability. Furthermore, the relatively narrow IFA distribution of MaliVD indicates that its performance is consistently stable across various vulnerability types, minimizing the impact of specific vulnerabilities or code structures. While LineVul excels at processing sequential information, its limitations in capturing complex control flow and data dependencies result in a higher false positive rate. IVDetect, despite leveraging graph-based representations, may introduce noise or lose critical information during PDG construction, contributing to false alarms. Furthermore, the lack of fine-grained output from GNNExplainer hinders the precise localization of vulnerable lines, leading to a higher IFA value. CPPCheck's reliance on predefined rules makes it susceptible to code variations and obfuscation, resulting in a high number of false positives. In comparison, MaliVD's multi-modal attention mechanism enables a deep understanding of code semantics and structure, coupled with targeted learning of vulnerability PoIs. This combination allows for more accurate identification of potentially vulnerable lines.

3) *Conclusion for RQ2:* The experimental results on both Top-k accuracy and IFA demonstrate MaliVD's superior effectiveness in localizing vulnerabilities at the code line level. By leveraging a multi-modal attention mechanism and focusing on vulnerability PoIs, MaliVD achieves higher accuracy and lower false alarm rates compared to LineVul, IVDetect, and CPPCheck. These findings highlight the practical utility and advancement of MaliVD in the field of vulnerability detection and localization, offering valuable assistance to security analysts in their efforts to identify and remediate software vulnerabilities.

C. RQ3: How Does MaliVD's Detection Performance Vary Across Vulnerability Categories?

To gain a deeper understanding of the models' classification capabilities for specific vulnerabilities, we visualize the IFA values of MaliVD, LineVul, IVDetect, and CPPCheck across six vulnerability categories in Figure 4. The box plots summarize the statistical properties of the IFA distributions, while the scatter plots provide insights into the dispersion of individual data points.

1) *Vulnerability-Specific Analysis:* For buffer overflow, MaliVD outperforms other methods, with a more concentrated IFA distribution centered closer to 1, indicating accurate vulnerability localization. Buffer overflow typically involve memory allocation and access through system functions like `strcpy` or `gets`. While CPPCheck and LineVul identify these functions, they generate many false positives due to insufficient context understanding. In contrast, MaliVD leverages PoIs to focus on code segments likely to trigger vulnerabilities, significantly reducing false alarms.

In value computation vulnerabilities, LineVul performs slightly better than MaliVD. Value computation vulnerabilities, such as integer overflows or division by zero, require precise numerical handling. Neural network-based models often struggle with numerical standardization. Although MaliVD faces similar challenges, its multi-modal attention mechanism helps it achieve competitive results.

Resource management vulnerabilities, such as double frees and memory leaks, involve complex conditional logic and dependencies. Static analysis tools like CPPCheck perform poorly here, as do neural network models like IVDetect.

MaliVD excels by integrating structural information from sequences, trees, and graphs, enabling it to accurately identify risky resource management patterns and reduce false positives.

Input validation issues arise when programs fail to properly sanitize user inputs. CPPCheck struggles with dynamic input scenarios, while MaliVD achieves superior performance by focusing on PoIs related to external inputs and potentially dangerous operations (e.g., database queries). This approach results in lower IFA values and higher stability compared to other methods.

Permission Management Vulnerabilities: These vulnerabilities involve complex access control logic, making them difficult to detect. All models exhibit high IFA values in this category, but MaliVD maintains the lowest median and most concentrated distribution. Its ability to integrate multiple code representations allows it to effectively capture intricate dependency patterns, surpassing the capabilities of traditional static analysis tools.

Command execution vulnerabilities are highly critical, often leading to severe consequences such as privilege escalation or system compromise. Detecting these vulnerabilities is challenging due to their reliance on string manipulation and dynamic inputs. MaliVD demonstrates robust performance, with significantly lower IFA values than other methods, attributed to its multi-modal attention mechanism that focuses on key code segments and PoIs.

2) *Conclusion for RQ3:* Overall, MaliVD demonstrates consistent and superior performance across all six vulnerability categories. Its multi-modal attention mechanism, combined with PoIs-based feature extraction, enables it to handle diverse vulnerability patterns with high accuracy and low false positive rates. Compared to existing methods like LineVul, IVDetect, and CPPCheck, MaliVD achieves significant improvements, particularly in complex and high-risk vulnerability types. By integrating multiple code representations and leveraging advanced learning strategies, MaliVD sets a new benchmark in vulnerability detection, offering practical value for real-world applications.

D. RQ4: What Drives MaliVD's Superior Performance in Vulnerability Detection and Localization?

To investigate the contribution of individual components to MaliVD's overall performance, we conduct an ablation study. This analysis evaluates the impact of removing key modules, including the different code representations and the PoIs-based learning mechanism. We assess the performance using Precision, Recall, F_1 -score for vulnerability type classification, and Accuracy@10 for vulnerability localization. The results are summarized in Table IV.

The ablation study examines the following modules: (1) Original code sequence features (NCS) and the sequence neural network (WLJAN); (2) AST tree-based features and TreeLSTM; (3) CPG graph-based features and GGNN; and (4) PoIs-based learning (masked features) and feature concatenations. We also evaluate the performance of (5) using only the PoIs-masked features without the original features and feature concatenations.

TABLE IV
ABLATION STUDY RESULTS

Models	P	R	F_1	$Acc@10$
w/o ¹ NCS & WLJAN	0.63	0.74	0.68	0.44
w/o AST & TreeLSTM	0.62	0.72	0.67	0.41
w/o CPG & GGNN	0.54	0.59	0.57	0.26
w/o Masked features & PoIs	0.58	0.65	0.61	0.31
w/o Original features	0.52	0.56	0.54	0.33
MaliVD	0.69	0.77	0.73	0.54

¹ w/o = without.

1) *Role of Multimodal Representation Learning:* Multi-modal representation learning is one of the core innovations of MaliVD. The model extracts three features from source code: original code sequence, abstract syntax tree, and code property graph, and uses three advanced neural networks of corresponding structures for learning. To evaluate the contribution of each of these three code representations to the model, we removed NCS, AST, and CPG respectively, as well as the corresponding WLJAN, TreeLSTM, and GGNN models, with the experimental results shown in Table IV.

The experimental results show that removing any code feature and model leads to a decrease in all classification and localization metrics, indicating that the three types of features are indispensable for vulnerability detection and localization. The removal of graph-based representation learning has the greatest impact, with the F_1 -score decreasing by 0.16 and $Acc@10$ decreasing by 0.18, indicating that graph-based representations can provide more information for vulnerability detection. In fact, the code property graph can effectively retain complex dependencies in the code, such as data flow, control flow, and function call relationships. These dependencies are often key factors in vulnerability generation. Through GGNN's learning of CPG, MaliVD can better capture vulnerabilities that may be caused by these dependencies, so as to achieve better results in vulnerability detection and localization.

In addition to graph-based representation learning, removing AST tree-based representation learning also leads to a significant decrease in these metrics. Although CPG itself contains the information of the AST, the TreeLSTM model, as a model specifically designed for processing tree-structured data, is more suitable for learning AST features. Unlike the learning and propagation method of graph neural networks, TreeLSTM learns the content of child nodes in a recursive manner and propagates it to the root node, which can better retain key semantic information. In addition, after removing the NCS representation and WLJAN network, the model's F_1 -score decreased by 7% and $Acc@10$ decreased by about 20%, indicating that sequence features are still an important feature for vulnerability detection in source code, which provides the "language characteristics" of the code and preserves the execution order and key statements of the code, which is also crucial for vulnerability detection and localization.

2) *Joint Effect of PoIs Features and Original Code Features:* The introduction of PoIs features enables MaliVD to have a stronger ability to judge vulnerability types and locate

vulnerabilities. To verify the role of PoIs features, we remove the PoIs features and only use the original three types of code features. Of course, the part of the PoIs mask and the part connected to the original features in the original model are also removed to see the performance change of the model. From the results, after removing PoIs, masked features, and feature concatenation, the model's $Acc@10$ decreases significantly, reaching only 0.31, and the model's precision, recall, and F_1 -score also decrease to a large extent. Compared with the results in Table III, *MalivD* loses its competitiveness after losing these key components.

To further verify the joint effect of PoIs features and original code features, we only use the PoIs masked part without using the original three types of code features. Interestingly, removing masked features and PoIs (0.31) and removing original features (0.33) leads to roughly similar $Acc@10$ decreases, and even more decreases in precision, recall, and F_1 -score, with the F_1 -score being only 0.54. This can actually be explained from the following two aspects: on the one hand, PoIs not only provide the type information of vulnerabilities, but also help the model exclude most of the code that is not related to vulnerabilities, making the model more focused on learning the locations where vulnerabilities may occur, which plays a great role in improving the location determination of vulnerabilities; on the other hand, although PoIs features are very important, they are only a small part of the original code (as mentioned earlier, the number of PoIs in most samples is less than 50), so if only PoIs features are used and the original complete code is removed, the model will not be able to learn the important context information in the original code, and only focusing on local information and losing global information will make the model lose the overall code understanding and vulnerability pattern learning. For example, for a statement that may trigger a vulnerability, it is labeled as a PoIs feature, but the program also makes sufficient conditional restrictions and dangerous operation filtering on this statement in other code parts, then this statement is likely not to have a vulnerability. If only the key code part is retained, then the model is very likely to judge this statement as having a vulnerability, thus causing false positives.

3) *Conclusion for RQ4*: The ablation experiments fully demonstrate the importance of each component in the *MalivD* model. The three code representations of sequence, tree, and graph complement each other, and the combination of PoIs features and the concatenation of original code features jointly improve *MalivD*'s performance in vulnerability detection and localization. The design of the *MalivD* model simulates the thinking mode of human beings when performing vulnerability detection and localization, they first understand the code content according to the characteristics of the original code (corresponding to the NCS and WLJAN models) combined with the syntax of the code (corresponding to the AST and TreeLSTM models), and then extract the key data flow and control flow (corresponding to the CPG and GGNN models) for in-depth analysis of the locations where vulnerabilities may be triggered (corresponding to the PoIs features) to determine the type and triggering location of the vulnerabilities.

VI. DISCUSSION

A. Threats to Validity

1) *External Validity*: Our experiments use three datasets, Devign, DiverseVul, and our newly constructed ReliVul. Devign is smaller but carefully curated; DiverseVul is larger with greater data imbalance and noise; ReliVul emphasizes real-world code, CWE labels, and precise trigger locations. While this provides coverage across different data regimes, generalization beyond these datasets may vary. We report results on all three, but future work should validate on additional ecosystems and programming languages.

2) *Internal Validity*: The line-level ground truth of ReliVul may carry bias from the labeling process. We mitigate this by constraining candidates via PoIs and requiring agreement across different techniques, which improves precision but does not eliminate all errors. Accordingly, we remove ambiguous cases and enforce PoI consistency before admitting a trigger line. In addition, low-frequency CWEs, namely uncommon vulnerability types, are not included in the ReliVul dataset.

B. Robustness of Feature Extraction

PoIs are defined at the effect level rather than by literal API names, and we apply PoI masks across sequence, AST, and CPG views. As the CPG captures data/control flows and call relations, equivalent behaviors invoked via wrappers or function pointers can still be recognized when their semantics are visible in the graph. Limitations remain if indirect calls are unresolved or wrappers substantially alter flows. We mitigate this with alias-aware analysis [55] to enrich the dataflow graphs. We exclude intentionally obfuscated code from our problem scope because it is uncommon in source-code level analysis and requires a different toolchain.

C. Theoretical Motivation

Applying a PoI mask and renormalization is equivalent to conditioning attention on the event of PoI-eligible lines. Thus PoI masking implements a hard prior that rules out non-causal regions and concentrates on semantically plausible triggers. It brings two benefits: (i) Sample efficiency. By reducing the effective hypothesis space, the model requires less time to perform inference. (ii) Optimization focus. Gradients flow to a smaller subset, sharpening the score margin between true triggers and noise. The three modalities provide complementary views, sequence, structure and dataflow, that improve *MalivD*'s ability to resolve challenging cases such as indirect calls and wrapped functions.

VII. CONCLUSION AND FUTURE WORK

In this paper, we present *MalivD*, a novel approach for vulnerability detection and localization in source code based on multi-modal attention learning. *MalivD* effectively integrating sequential, tree-based, and graph-based representations of source code, allowing it to capture a more comprehensive understanding of code semantics and structure. By strategically focusing on PoIs within the code, *MalivD* enhances its ability to precisely locate vulnerabilities. *MalivD* outperforms

SOTA methods in both vulnerability detection and localization on three large vulnerability datasets. Ablation studies further confirm the importance of each component within MaliVD.

In the future, we will explore more vulnerability patterns and extend MaliVD to support other programming languages. Furthermore, we will evaluate MaliVD in real-world projects and integrate it into practical security workflows.

REFERENCES

- [1] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.
- [2] 2024 Annual Cybersecurity Vulnerability Threat Situation Research Report, QiAnXin Group, Beijing, China, 2025.
- [3] T. D. LaToza, G. Venolia, and R. DeLine, "Maintaining mental models: A study of developer work habits," in *Proc. 28th Int. Conf. Softw. Eng.*, May 2006, pp. 492–501.
- [4] CodeQL. Accessed: Mar. 5, 2025. [Online]. Available: <https://codeql.github.com>
- [5] Coverity: Coverity Scan Static Analysis. Accessed: Feb. 10, 2025. [Online]. Available: <https://scan.coverity.com>
- [6] Cppcheck. Accessed: Jan. 17, 2025. [Online]. Available: <http://cppcheck.net>
- [7] Flawfinder. Accessed: Mar. 5, 2025. [Online]. Available: <https://dwtwheeler.com/flawfinder>
- [8] D. Andrew. *Rough-Auditing-Tool-for-Security*. Accessed: Jan. 17, 2025. [Online]. Available: <https://github.com/andrew-d/rough-auditing-tool-for-security>
- [9] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "μvuldeepecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2224–2236, Sep. 2019.
- [10] R. Russell et al., "Automated vulnerability detection in source code using deep representation learning," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Aug. 2018, pp. 757–762.
- [11] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 1–11.
- [12] X. Li, L. Wang, Y. Xin, Y. Yang, and Y. Chen, "Automated vulnerability detection in source code using minimum intermediate representation learning," *Appl. Sci.*, vol. 10, no. 5, p. 1692, Mar. 2020.
- [13] M. Zagane, M. K. Abdi, and M. Alenezi, "Deep learning for software vulnerabilities detection using code metrics," *IEEE Access*, vol. 8, pp. 74562–74570, 2020.
- [14] H. Wang et al., "Combining graph-based learning with automated data collection for code vulnerability detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1943–1958, 2021.
- [15] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A framework for using deep learning to detect software vulnerabilities," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2244–2258, Jul. 2022.
- [16] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2021, pp. 292–303.
- [17] W. Wang, T. N. Nguyen, S. Wang, Y. Li, J. Zhang, and A. Yadavally, "DeepVD: Toward class-separation features for neural network vulnerability detection," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 2249–2261.
- [18] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 654–668.
- [19] Y. Mirsky et al., "VulChecker: Graph-based vulnerability localization in source code," in *Proc. 32nd USENIX Secur. Symp. (USENIX Security)*, 2023, pp. 6557–6574.
- [20] W. Zhou et al., "The security of using large language models: A survey with emphasis on ChatGPT," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 1, pp. 1–26, Jan. 2025.
- [21] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, Feb. 2025.
- [22] X. Feng, X. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, "Detecting vulnerability on IoT device firmware: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2023.
- [23] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surveys*, vol. 54, no. 11s, pp. 1–36, Sep. 2022.
- [24] A. Hindle, E. T. Barr, M. Gabel, Z. Su, and P. Devanbu, "On the naturalness of software," *Commun. ACM*, vol. 59, no. 5, pp. 122–131, 2016.
- [25] H. Perl et al., "VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*. New York, NY, USA: Association for Computing Machinery, 2015, pp. 426–437.
- [26] S. Cao, X. Sun, L. Bo, Y. Wei, and B. Li, "BGNN4 VD: Constructing bidirectional graph neural-network for vulnerability detection," *Inf. Softw. Technol.*, vol. 136, Aug. 2021, Art. no. 106576.
- [27] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "VulDeeLocator: A deep learning-based fine-grained vulnerability detector," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2821–2837, Jul. 2022.
- [28] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "VulCNN: An image-inspired scalable vulnerability detection system," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 2365–2376.
- [29] V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, and D. Phung, "ReGVD: Revisiting graph neural networks for vulnerability detection," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng., Companion (ICSE-Companion)*, May 2022, pp. 178–182.
- [30] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 1456–1468.
- [31] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction," in *Proc. IEEE/ACM 19th Int. Conf. Mining Softw. Repositories (MSR)*, Pittsburgh, PA, USA, May 2022, pp. 608–620.
- [32] D. Hin, A. Kan, H. Chen, and M. A. Babar, "LineVD: Statement-level vulnerability detection using graph neural networks," in *Proc. IEEE/ACM 19th Int. Conf. Mining Softw. Repositories (MSR)*, May 2022, pp. 596–607.
- [33] C. Zhang, B. Liu, Y. Xin, and L. Yao, "CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation," *IEEE Trans. Softw. Eng.*, vol. 49, no. 8, pp. 4152–4168, Aug. 2023.
- [34] P. Xiao, Q. Xiao, X. Zhang, Y. Wu, and F. Yang, "Vulnerability detection based on enhanced graph representation learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5120–5135, 2024.
- [35] F. Qiu, Z. Liu, X. Hu, X. Xia, G. Chen, and X. Wang, "Vulnerability detection via multiple-graph-based code representation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 8, pp. 2178–2199, Aug. 2024.
- [36] H.-C. Tran, A.-D. Tran, and K.-H. Le, "DetectVul: A statement-level code vulnerability detection for Python," *Future Gener. Comput. Syst.*, vol. 163, Feb. 2025, Art. no. 107504.
- [37] A. Mechri, M. A. Ferrag, and M. Debbah, "SecureQwen: Leveraging LLMs for vulnerability detection in Python codebases," *Comput. Secur.*, vol. 148, Jan. 2025, Art. no. 104151.
- [38] D. Zou, Y. Zhu, S. Xu, Z. Li, H. Jin, and H. Ye, "Interpreting deep learning-based vulnerability detector predictions based on heuristic searching," *ACM Trans. Softw. Eng. Methodology*, vol. 30, no. 2, pp. 1–31, Apr. 2021.
- [39] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 9240–9251. [Online]. Available: <https://dblp.org/rec/conf/nips/YingBYZL19.html>
- [40] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [41] Y. Mao, Y. Li, J. Sun, and Y. Chen, "Explainable software vulnerability detection based on attention-based bidirectional recurrent neural networks," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 4651–4656.
- [42] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.
- [43] B. Hui et al., "Qwen2.5-coder technical report," 2024, *arXiv:2409.12186*.
- [44] B. Steenhoeck, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 2237–2248.
- [45] Joern. Accessed: Jan. 17, 2025. [Online]. Available: <https://joern.io>

- [46] *Cwe Platform*. Accessed: Jan. 17, 2025. [Online]. Available: <https://cwe.mitre.org/>
- [47] X. Xiao et al., “RBLJAN: Robust byte-label joint attention network for network traffic classification,” *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2161–2178, May 2025.
- [48] K. S. Tai, R. Socher, and C. D. Manning, “Improved semantic representations from tree-structured long short-term memory networks,” 2015, *arXiv:1503.00075*.
- [49] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, “Gated graph sequence neural networks,” 2015, *arXiv:1511.05493*.
- [50] D. Tang, B. Qin, and T. Liu, “Document modeling with gated recurrent neural network for sentiment classification,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Lisbon, Portugal, 2015, pp. 1422–1432.
- [51] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.
- [52] *NVD: National Vulnerability Database*. Accessed: Jan. 17, 2025. [Online]. Available: <https://nvd.nist.gov>
- [53] *Mend*. Accessed: Jan. 17, 2025. [Online]. Available: <https://www.mend.io>
- [54] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, “Code2vec: Learning distributed representations of code,” *Proc. ACM Program. Lang.*, vol. 3, no. POPL, pp. 1–29, Jan. 2019.
- [55] J. Späth, K. Ali, and E. Bodden, “IDE AI: Efficient and precise alias-aware dataflow analysis,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, pp. 1–27, Oct. 2017.



Enze Dai received the B.Eng. degree from Central South University, Changsha, in 2023. He is currently pursuing the M.Eng. degree with Shenzhen International Graduate School, Tsinghua University. His research interests include software testing and software security. His technical interests include fuzzing and deep learning.



Shuo Wang received the B.Eng. degree in cyberspace security from Beijing University of Posts and Telecommunications in 2022. He is currently pursuing the M.Eng. degree with Shenzhen International Graduate School, Tsinghua University. His research interests include machine learning, information security, and computer networks.



Xi Xiao received the Ph.D. degree from the State Key Laboratory of Information Security, Graduate University of Chinese Academy of Sciences, in 2011. He is currently an Associate Professor with Shenzhen International Graduate School, Tsinghua University. His research interests include information security and the computer networks.



Qing Li (Senior Member, IEEE) received the B.S. degree in computer science and technology from Dalian University of Technology in 2008 and the Ph.D. degree in computer science and technology from Tsinghua University in 2013. He is currently an Associate Researcher with the Peng Cheng Laboratory, Shenzhen, China. His research interests include network function virtualization, in-network caching/computing, and intelligent self-running networks.



Sheng Wen (Senior Member, IEEE) received the Ph.D. degree from Deakin University, Australia, in 2014. He is currently an Associate Professor with Swinburne University of Technology. He is also the Director of the Blockchain Innovation Laboratory and the Deputy Director of Swinburne Cybersecurity Laboratory, Swinburne University. In addition, he has published more than 100 high-quality articles, including top conference papers, such as papers in IEEE S&P, ACM CCS, USENIX Sec, NDSS, ICSE, FSE, and WWW, as well as many articles in

IEEE/ACM TRANSACTIONS series journals. Some of these articles are with high impact factors or become highly cited articles. His research interests include social network analysis and system security.



Tianqing Zhu (Member, IEEE) received the Ph.D. degree in computer science from Deakin University, Australia, in 2014. She is currently a Professor and the Associate Dean (Research) with the Faculty of Data Science, City University of Macau. She has published extensively in top-tier journals and conferences, including IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and *ACM*

Computing Surveys. She has also co-authored books, such as *Differential Privacy and Applications* and *Location Privacy in Mobile Applications*. Her research interests include privacy-preserving techniques, network security, and data mining.